



BSc (Hons) in **Information Technology**  
**Specialized in AI**

**IT3012 : Intelligent Agents**

Year 3 · Semester 1 · 2026 Semester 2

**Faculty of Computing**

## Practical 03

**Objective & Lecture Mapping:** In previous labs, we built a Simple Reflex Agent that moved randomly or reacted only to immediate adjacent cells. Today, we transition to a **Goal-Based/Planning Agent**. You will expose the environment's state space to the agent, allowing it to use **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, and **Uniform-Cost Search (UCS)** to simulate and find paths to the food before taking a physical action.

### Part 1: Practical Implementation — Building the Search Agent

#### Step 1.1: Exposing the World Model

Classical search algorithms require an abstract model of the environment to simulate future states. Currently, your agent is "blind" to the overall map.

1. Create a new Branch called Week\_03 in your Forked Github Repo and work on the below Questions.
2. Open `visual_grid_game.py` .
3. Locate the `get_percept(self)` method.
4. Add three new keys to the dictionary to pass the global state to the agent:  

```
'grid_size': (self.width, self.height)
'walls': list(self.walls)
'all_food': list(self.food_positions)
```

#### Step 1.2: Implementing BFS, DFS, and UCS

You will implement three distinct uninformed search strategies. The core node expansion structure is identical; only the data structure for the **Frontier** changes!

1. Open `agent.py` and create a new class called `SearchAgent`.
2. Import required structures: `from collections import deque` and `import heapq`.
3. **BFS:** Create `def bfs_search(...)`. Use a FIFO queue (`deque.popleft()`). This explores the shallowest nodes first.
4. **DFS:** Create `def dfs_search(...)`. Use a LIFO stack (`list.pop()`). This explores the deepest nodes first.
5. **UCS:** Create `def ucs_search(...)`. Use a Priority Queue (`heapq.heappop()`) ordered by the total path cost  $g(n)$ .
6. For all three algorithms, you must maintain a `reached` set (or visited array) to track explored states. This converts your algorithm from a *Tree Search* to a *Graph Search*, preventing infinite loops.

### Step 1.3: Executing and Comparing Offline Plans

The agent must now form a complete plan and execute it step-by-step.

1. In your `SearchAgent.__init__`, add an empty list: `self.plan = []` and a config string `self.active_algo = 'BFS'`.
2. In `sense_and_act(self, percept)`, check if `self.plan` is empty.
3. If empty, find the closest food pellet from `percept['all_food']`, execute the search method matching `self.active_algo`, and store the resulting sequence of actions in `self.plan`.
4. Return the first action from the plan using `return self.plan.pop(0)`.
5. **Observation Task:** Change `self.active_algo` between 'BFS', 'DFS', and 'UCS' and run the simulation. Watch how DFS takes erratic, winding paths, while BFS and UCS take direct, optimal paths!

## Part 2: Theoretical Evaluation

Based on your implementation and Lecture 04, complete the following analytical questions.

**1. (Remember) You built your search logic based on the environment coordinates. What is the fundamental difference between the "State Space" and the "Search Tree"?**

~~State Space in intelligent system is a fundamental concept used to model and solve problems by representing all possible configurations of a system. It is widely applied in artificial intelligent, game theory just like we use in this game(DFS,BFS,UCS search agent). Here, the nodes are the non-wall grid cells (x, y), and the edges represent movements up, down, left, and right.

~~Search Tree is a fundamental technique in AI, used to explore possible sequences of actions to find a path from a start state to a goal state: the starting cell serves as the root, and each expansion adds child nodes. The same cell may appear multiple times in the tree. once for every path by which it is reached. meaning the tree can be significantly larger than the state space.

**2. (Understand) In Step 1.2, you were instructed to use a `reached` set. In graph search theory, what specific problem does this set solve, and what would happen to your DFS agent without it?**

The reached set records states that have already been expanded, then the algorithm never expands the same cell twice. It turning over tree search into graph search and prevents redundant paths and loops. without it, my DFS could go left,right,left,right between 2 cells forever. The grid is finite, but search would never end and agent would never reach to the food. each cell was expanded at most once, as a example 35 nodes for a 34 move plan.

**3. (Analyze) Compare the visual paths generated by BFS and DFS in your simulation. Why is BFS guaranteed to be optimal in this specific grid, whereas DFS produces winding, suboptimal routes?**

In my DFS run the paths wound all over the grid. The 1st food was 6 cells away, but DFS took 34 moves to get there. The food at (7,3), a cell from the agent, took 3 jumps. Its plans were 27 to 99 moves long, and it used all 400 steps, ate only 7 of 15 food and scores 35, since the long paths also crossed hidden traps. BFS ate all 15 food in 59 steps, with plans averaging about 3 moves and a score of 35. UCS ate all 15 in 87 steps, with plans averaging about 5 moves and a

score of 285. BFS uses a FIFO queue and expands nodes level by level, so the first time it reaches the food it has used the fewest moves. With every move costing 1, that path is optimal, and UCS gives the same result. DFS uses a LIFO stack, goes as deep as it can along one branch and returns the first path it finds, not the shortest.

**4. (Evaluate) If we scaled `visual\_grid\_game.py` up to a massive 1000x1000 grid, BFS and UCS might crash before finding food. According to the time and space complexity formulas from the lecture, contrast the memory bottlenecks of BFS versus DFS.**

As we learnt in the lecture, BFS has time  $O(b^d)$  and space complexity  $O(b^d)$ , because it stores the whole frontier. UCS is similar, with  $O(b^{(1 + \lceil C^*/e \rceil)})$ , and with equal step costs it behaves like BFS. DFS has time complexity  $O(b^m)$  but space complexity only  $O(bm)$ , since it keeps just the current path and its siblings. My runs show BFS and UFS and UCS growth: the last UCS plan was only 21 moves but expanded 137 nodes, almost the whole 12,12 grid (139 free cells). On a 1000, 1000 grid that would mean about a million states in memory, which is why BFS and UCS can crash before finding food. DFS needs little memory but can search for a very long time and return a huge path, as its 99move plan showed. On the small grid DFS actually expanded more nodes than BFS, for example 107 against 2 for one plan, so its advantage is memory, not speed or path length.

**Once Completed Get a PDF of the completed File and Push into the Week 03 brach in the Github Project**

Finalize and Lock Lab 03 Documentation



**FACULTY OF COMPUTING**